Northeastern University
Electrical and Computer Engineering

College of Engineering

Department of Electrical and Computer Engineering

EECE 2140: COMPUTING FUNDAMENTALS FOR ENGINEERS

Spring 2026

Dr. Fatema Nafa

Mini Project #03

Mini Project 3: Copilot-Powered Image Denoising

Goal

Build a C++ program that "denoises" a grayscale image represented as a **2D matrix** by applying a **neighborhood filter**

You are not required to use real image formats. We'll treat an image as a grid of integers (0–255).

What you will build

A terminal program that:

1. Loads a matrix (image) from a text file (or generates a noisy one).
2. Applies a denoising filter using **3×3 neighborhood operations**.
3. Writes the denoised matrix to an output file.
4. Reports before/after statistics (optional but recommended).

Input/Output format (simple, required)

Use a plain text format:

- First line: rows cols
- Next rows lines: cols integers per line (0–255)

Example:

```
4 5
10 10 10 10 10
10 200 10 200 10
10 10 10 10 10
10 200 10 200 10
```

Required features

A) Matrix handling

- Store the image as vector<vector<int>> *or* dynamically allocated int**.
- Validate input ranges: clamp values to [0,255] after filtering.

B) Neighborhood filter (core requirement)

Implement at least **one** of the following:

- **Mean filter (3×3 average)**
- **Median filter (3×3 median)** (stronger denoising but slightly harder)

C) Border handling (must choose one and document it)

Pick one border policy:

1. **Clamp** (replicate edge pixels)
2. **Zero padding**
3. **Ignore borders** (only filter interior pixels; copy borders unchanged)

D) Denoising pass

- Apply the filter for **1 iteration** (required)
- Support k iterations as a command-line option (bonus)

E) Command-line interface (recommended)

Your program should support:

denoise <input.txt> <output.txt> <filter> <iterations>

Example:

denoise noisy.txt clean.txt mean 2

Testing requirements (mandatory)

Include at least **3 test cases** in a tests/ folder:

1. Small matrix where you can manually verify output.
2. A matrix with impulse noise (random “salt and pepper” pixels).
3. Edge/border behavior test (values near borders).

Also include a short TESTING.md explaining:

- How to run tests

- Expected outcomes

Copilot usage rules (required)

In README.md, add a short section:

- What Copilot helped with (e.g., parsing file, median logic)
- What you wrote/verified yourself
- One bug Copilot suggested or missed and how you fixed it

Deliverables

- GitHub repo with:
  - src/ code
  - tests/ input matrices
  - README.md (how to build/run + design notes)
  - TESTING.md
- A short demo screenshot or terminal output snippet in README